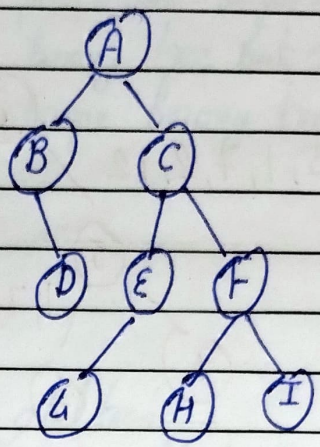
Date : / /
Page No.

Inorder, Preorder and Postorder traversal of a binary tree

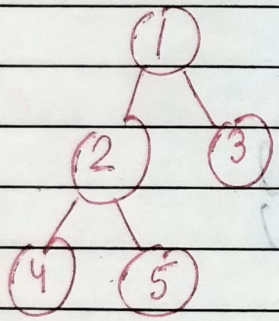
Inorder:- Left Root Right
Preorder- Root Left Right
Postorder- Left Right Root

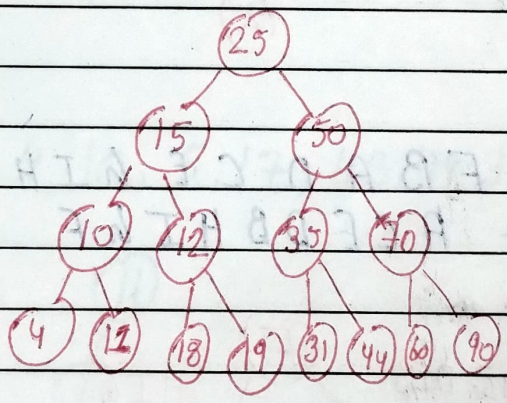


Inorder:-
BDAGECHFI

Preorder:- ABDCEGFHI

Postorder:- DBGEHIFCA



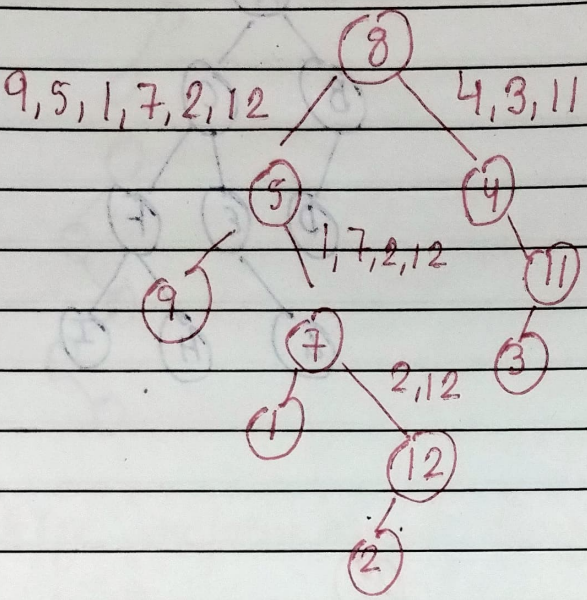


Preorder:- 1 2 4 5 3
Postorder:- 4 5 2 3 1
Inorder: 4 2 5 1 3

Postorder:- 4 11 10 18 19 12 15 31 44 35 60 90 70 50 25
Preorder:- 25 15 10 4 11 12 18 19 50 35 31 44 70 60 90
Inorder:- 4 10 11 15 18 12 19 25 31 35 44 50 60 70 90

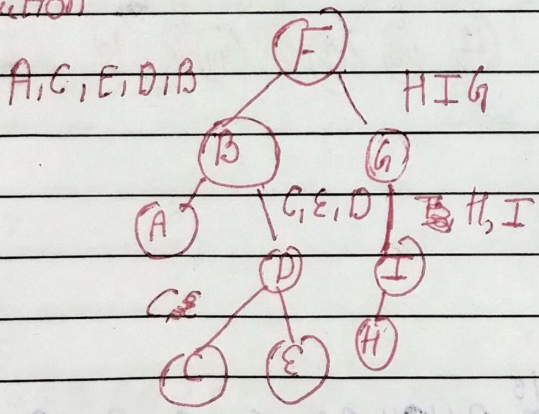
Date : / /
Page No.

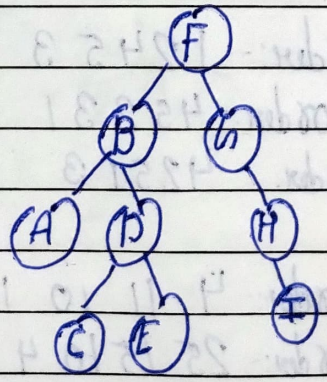
Postorder:- 9, 1, 2, 12, 7, 5, 3, 11, 4, 8 — L R Root
Inorder - 9, 5, 1, 7, 2, 12, 8, 4, 3, 11 — L Root R



Preorder:- F B A D C E G I H (Root L R)
Postorder - A C E D B H I G F (L R Root)

I Solution

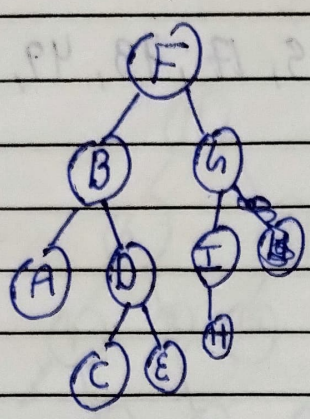




Preorder:- F B A D C E G I H
Postorder:- A C E D B H I G F

Date : / /
Page No.

II - Solution



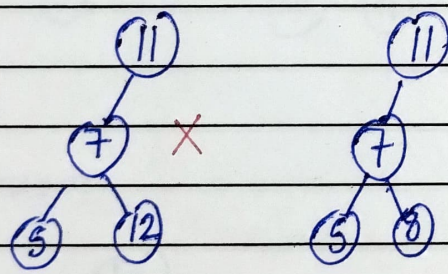
Using preorder and postorder unique full binary tree but not unique binary tree.

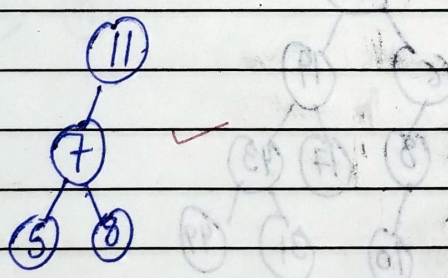
Binary Search Tree

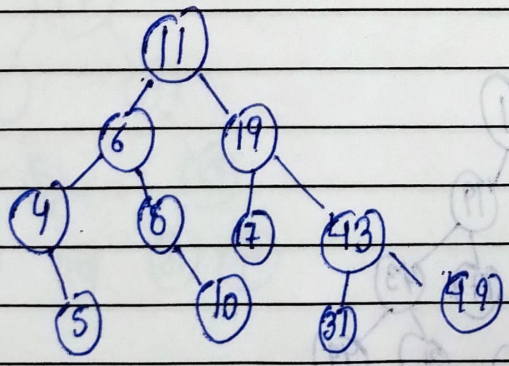
(1) Each node must have atmost 2 children
(2) The value of left node must be smaller than parent node and value of right node must be greater than parents.

Draw Binary Search tree

11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31





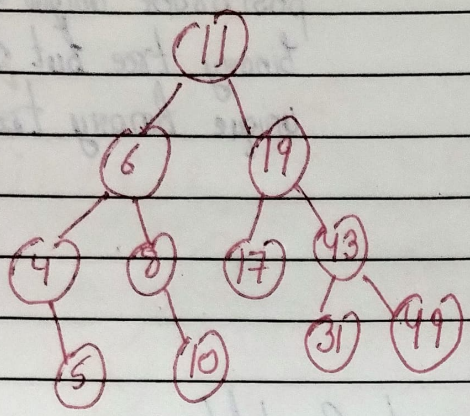


Date : / /
Page No.

Deletion in BST

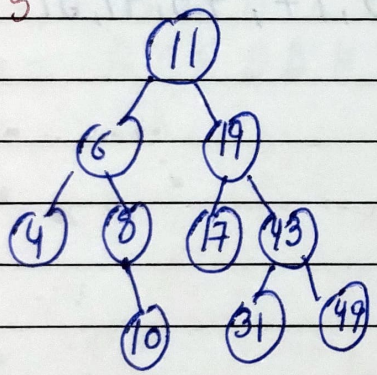
11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31

(1)

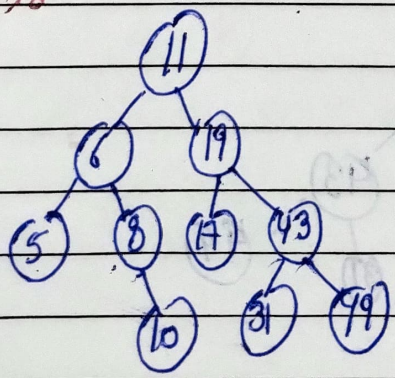


Case I No child
Case II 1 child
Case III 2 child

Case I No child
Delete 5



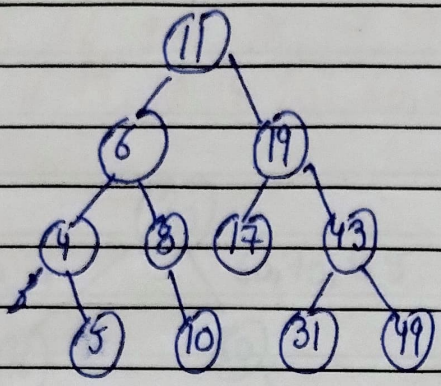
Case II 1 child
Delete 4



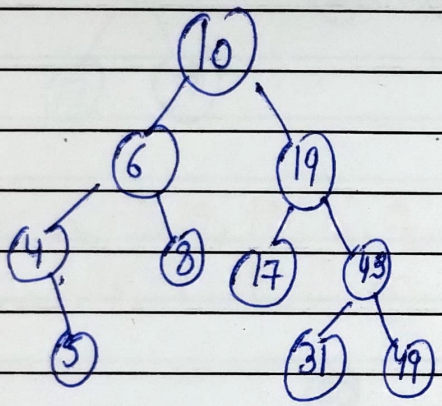
Date : / /
Page No.

Case III 2 child (i) Inorder Predecessor (ii) Inorder Successor

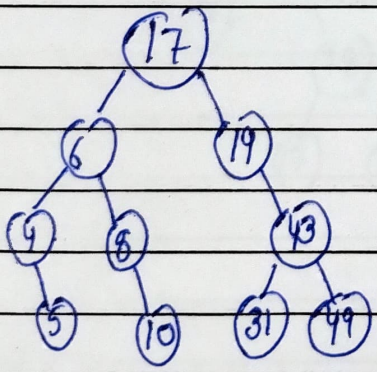
Delete (11)



Inorder Predecessor

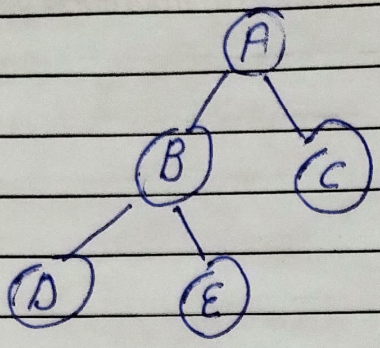


Inorder Successor

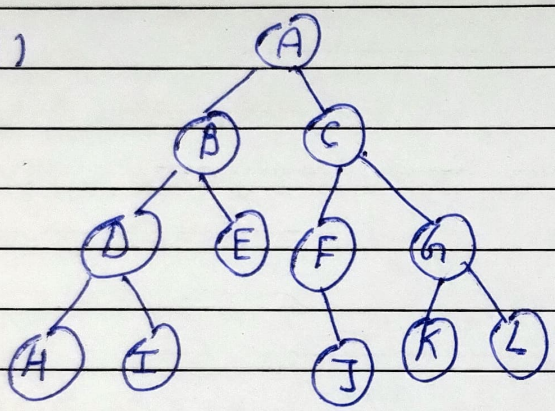


Date : / /
Page No.

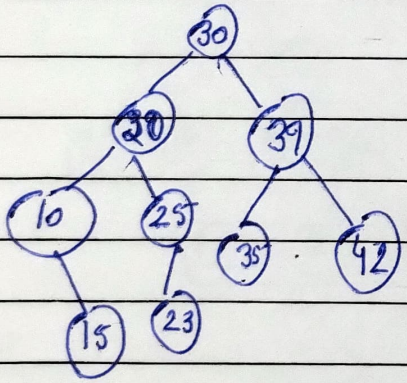
(1)



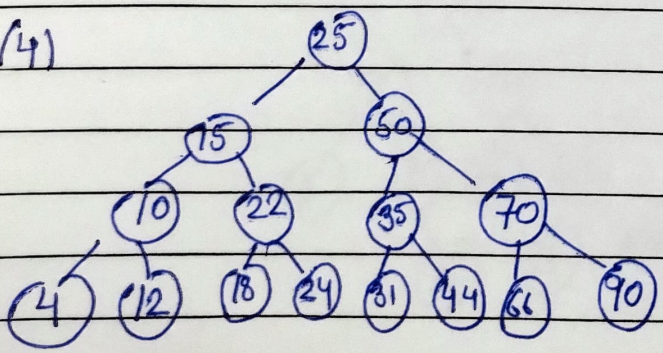
(2)



(3)



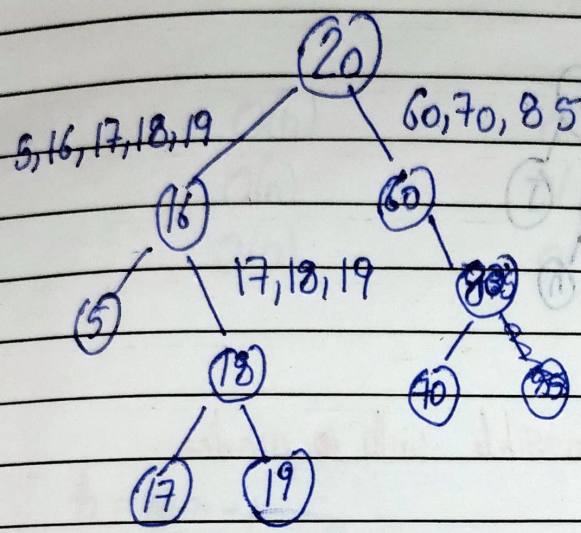
(4)



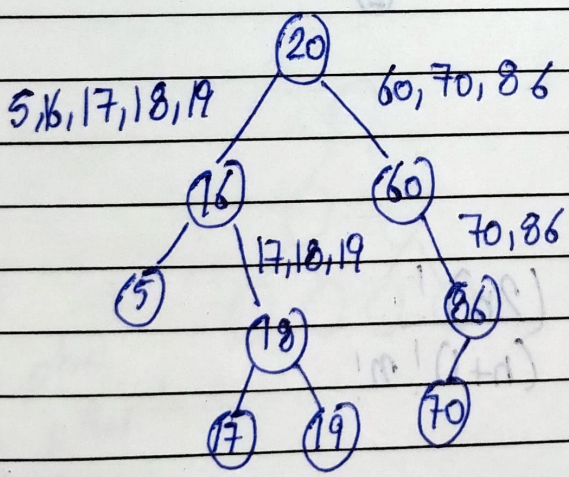
Date : / /
Page No.

Construction of BST when only preorder or postorder is given

Preorder :- 20, 16, 5, 18, 17, 19, 60, 85, 70 (Root left Right)
Inorder :- 5, 16, 17, 18, 19, 20, 60, 70, 85 (left Root Right)



Postorder :- 5, 17, 19, 18, 16, 70, 86, 60, 20 (left Right Root)
Inorder :- 5, 16, 17, 18, 19, 20, 60, 70, 86

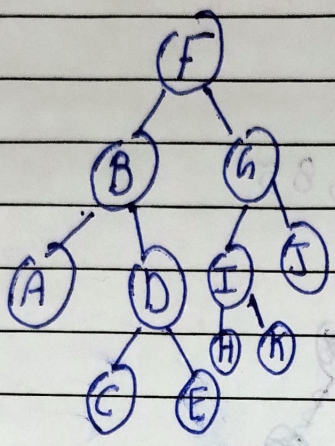


Date : / /
Page No.

Construct Binary tree from given preorder & postorder

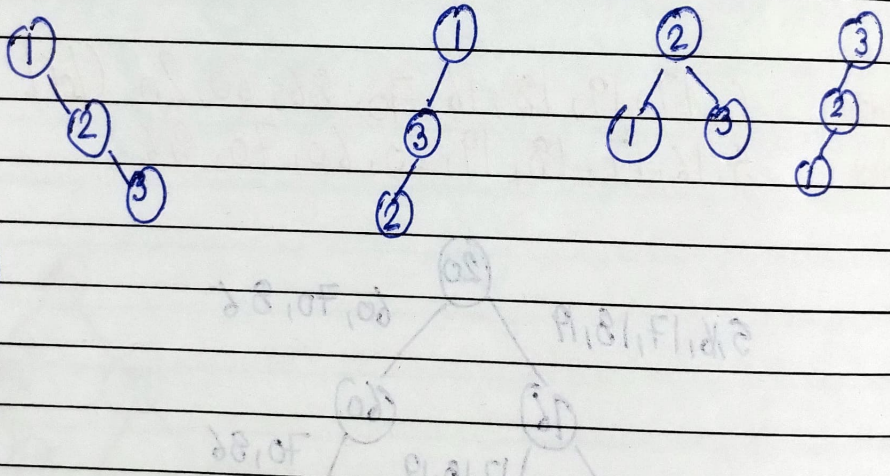
Preorder - F B A D C E G I H K J (Root L R)

Postorder - A C E D B H K J I G F (L R Root)



Binary ~~tree~~ Search tree possible with 3 nodes.

1, 2, 3



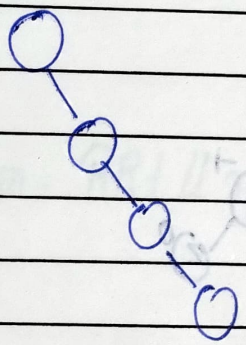
$$C_n = \frac{(2n)!}{(n+1)!\,n!}$$

Date : / /
Page No.

Time and space complexity.
Binary Search tree

Operation	Worst case	Average case	Best case	Space
Search	O(n)	O(logn)	~~O(logn)~~ O(1)	O(n)
Insert	O(n)	O(logn)	~~O(logn)~~ O(1)	O(n)
Delete	O(n)	O(logn)	O(logn)	O(n)

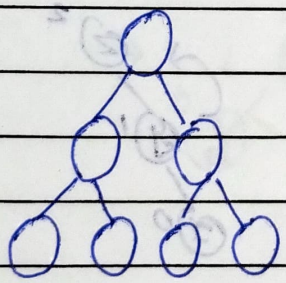
Worst case



$h = n-1$

Time complexity = O(n)

Average case



$n = 2^{h+1} - 1$

$n+1 = 2^{h+1}$

$\log_2(n+1) = (h+1)\log_2 2$

$h+1 = \log_2(n+1)$

$\boxed{h = \log_2(n+1) - 1}$

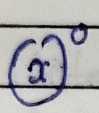
Date : / /
Page No.

AVL Tree

① It is a Binary Search tree.

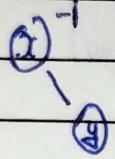
② Height of left Subtree - Height of right Subtree = {-1, 0, 1} called Balance factor.

Case I



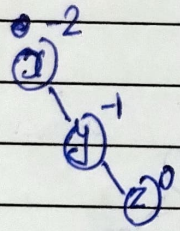
Balance tree

Case II



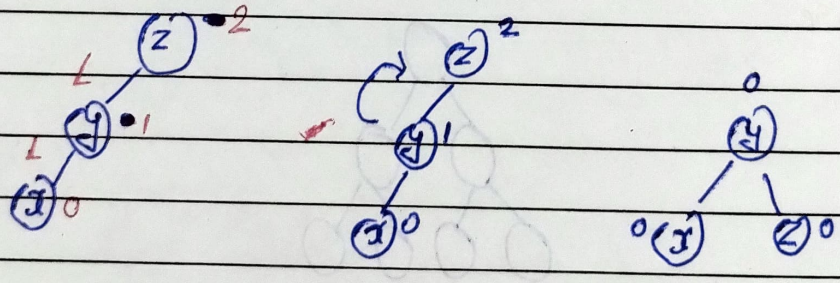
Balanced tree

Case III

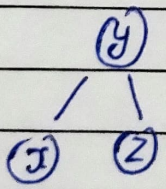


Unbalanced tree

(a) LL Rotation



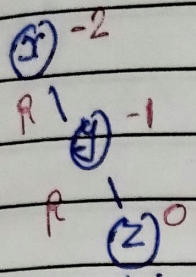
Clockwise rotation

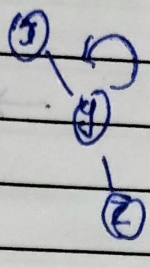


Balanced tree

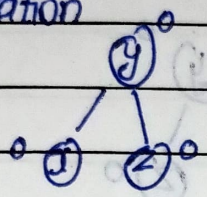
Date : / /
Page No.

(b) RR rotation

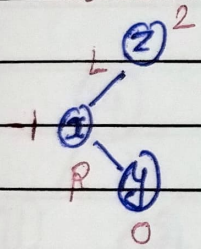




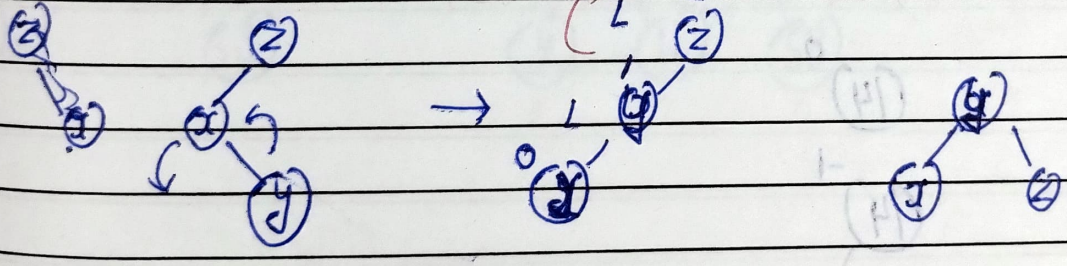
Anticlockwise rotation



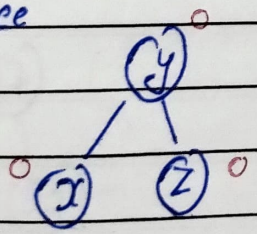
(c) LR rotation



LR rotation = RR + LL rotation

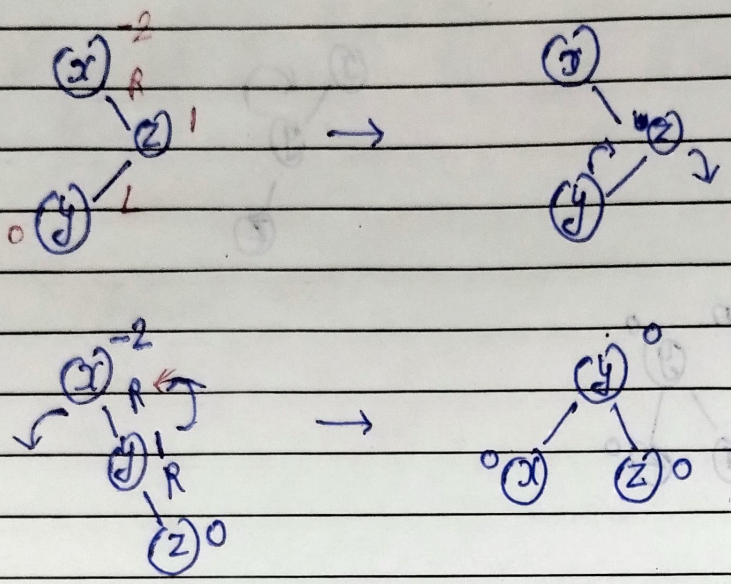


Balanced tree

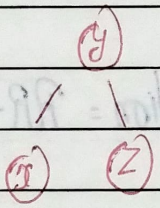


Date : / /
Page No.

(d) RL rotation

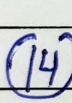


Balanced tree

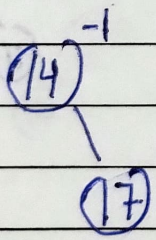


Eg- 14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20

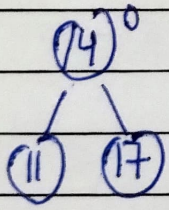
I



II

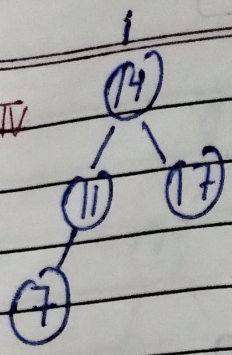


III

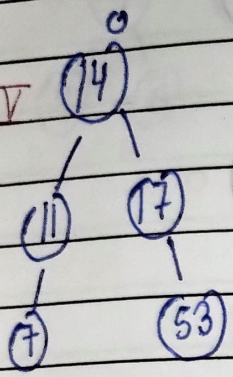


Date : / /
Page No.

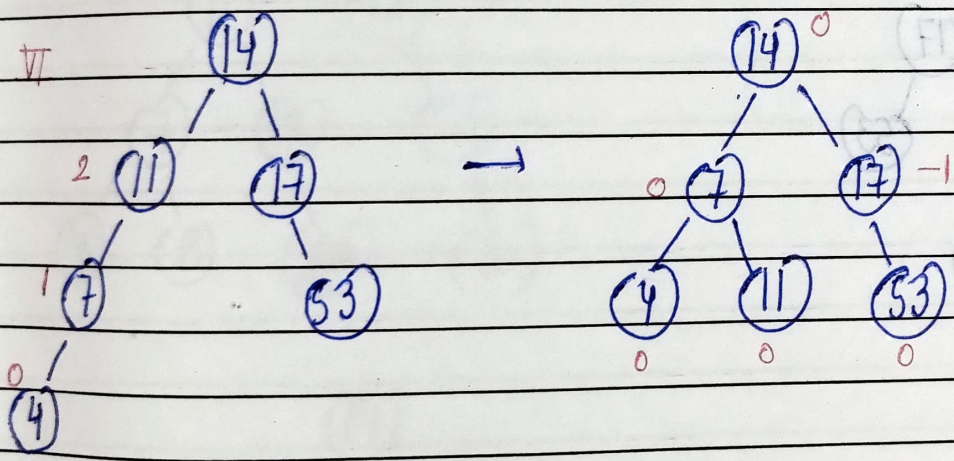
IV



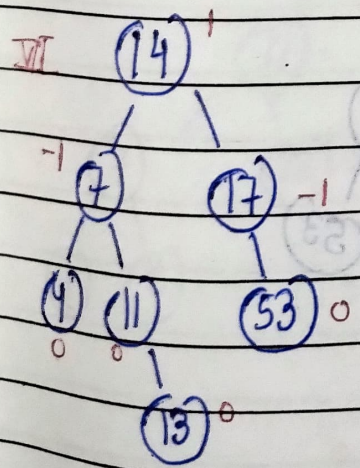
V



VI

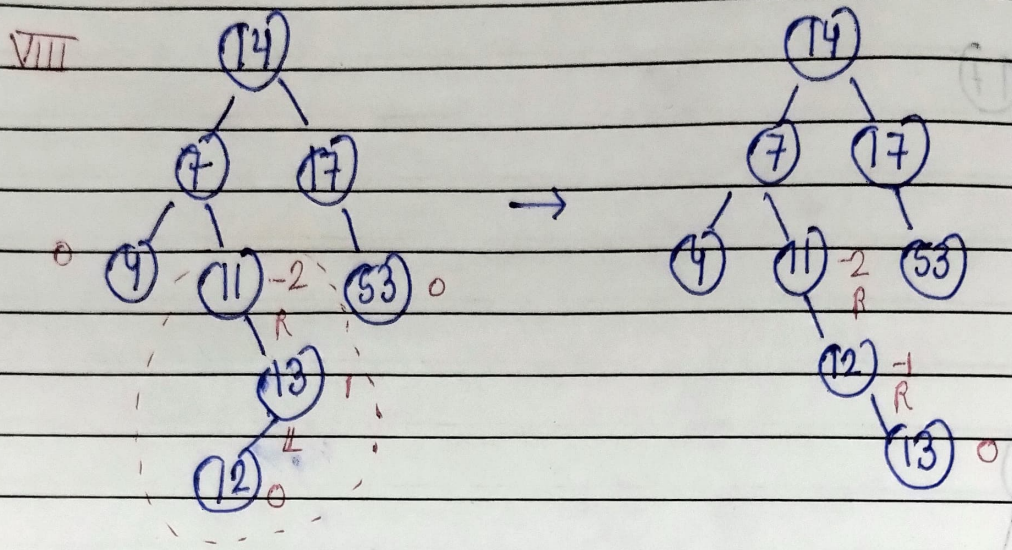


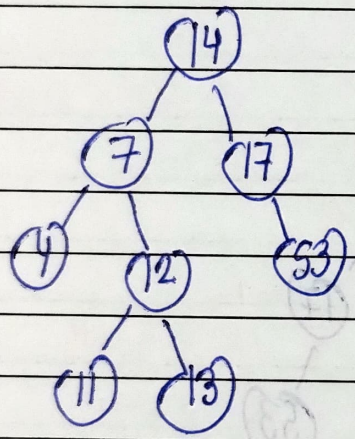
VII



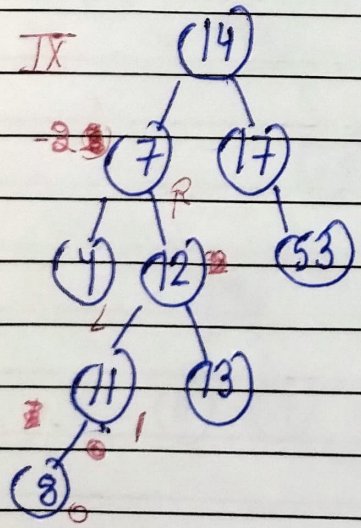
Date : / /
Page No.

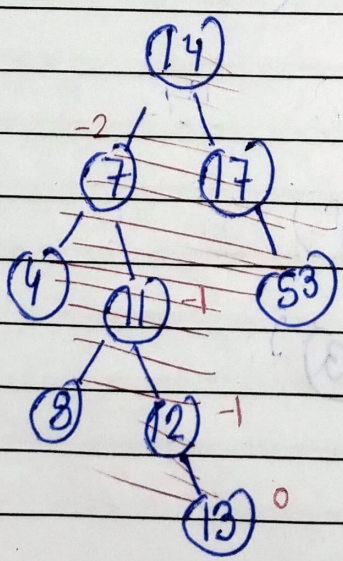
VIII



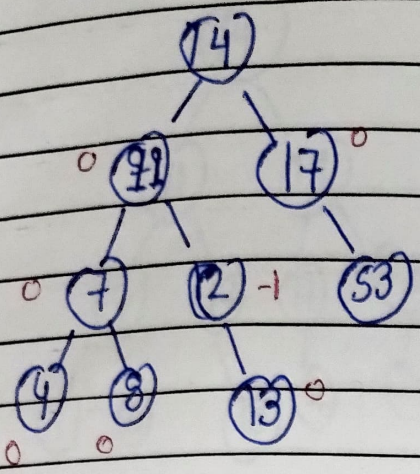


IX

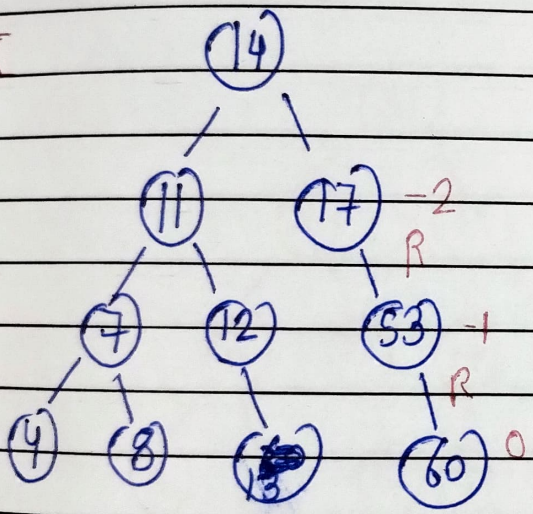




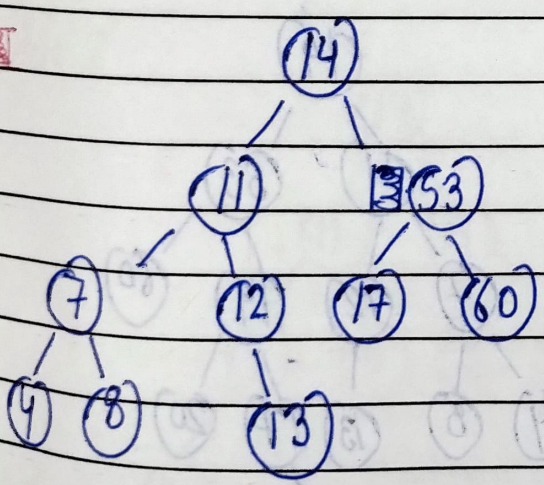
Date : / /
Page No.



X

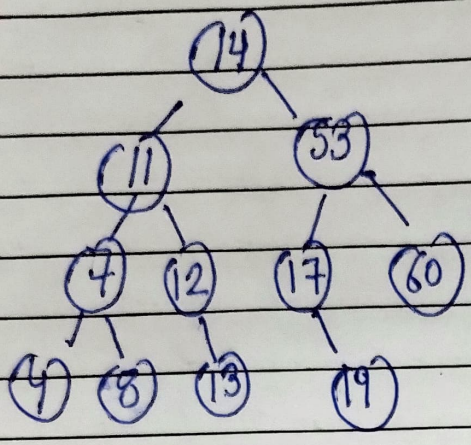


X

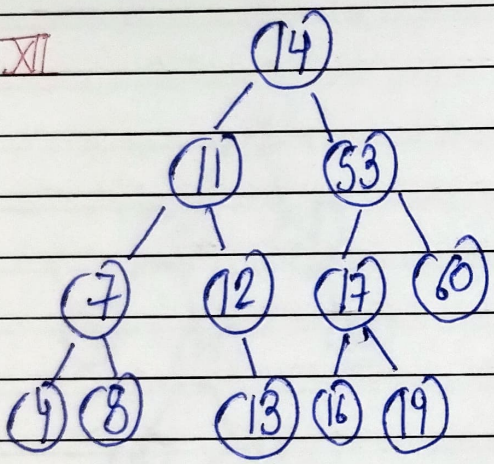


Date : / /
Page No.

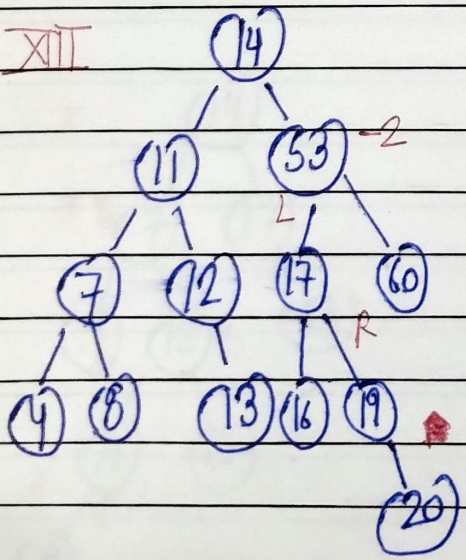
XI

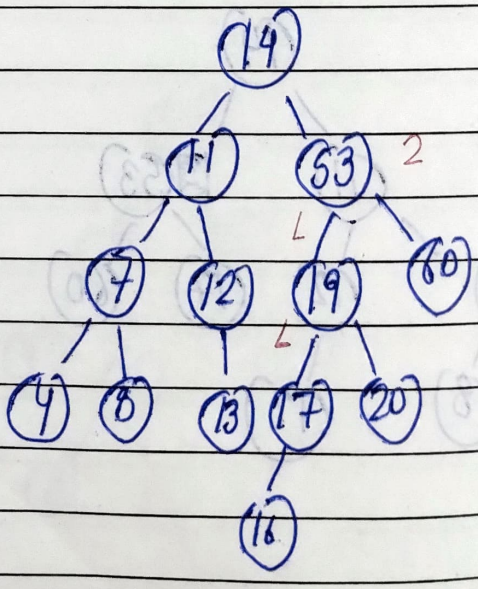


XII

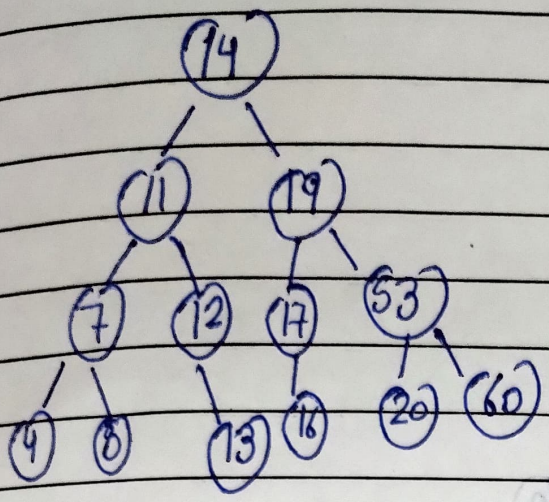


XIII



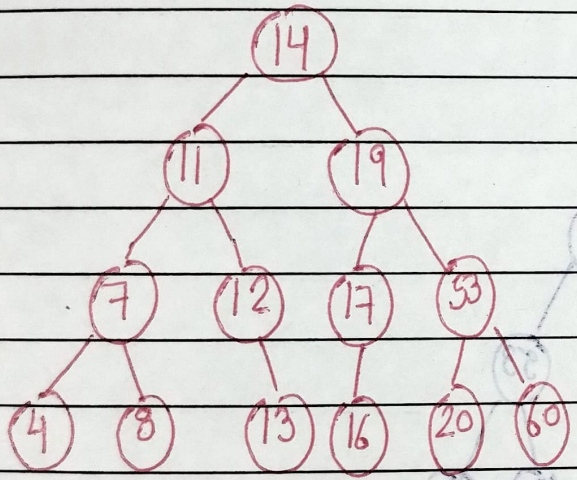


Date : / /
Page No.

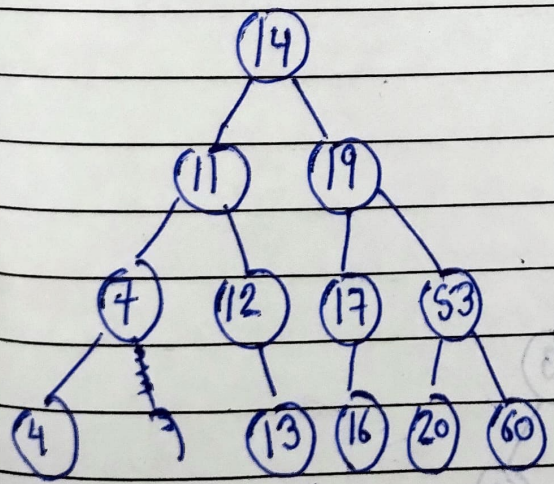


Deletion in AVL Tree

Step I Delete the node according to (BST cases)
Step II Calculate balance factor

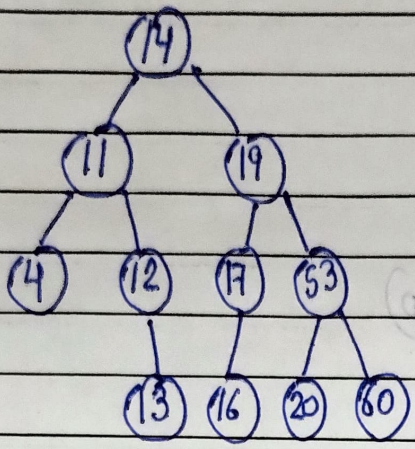


Delete 8

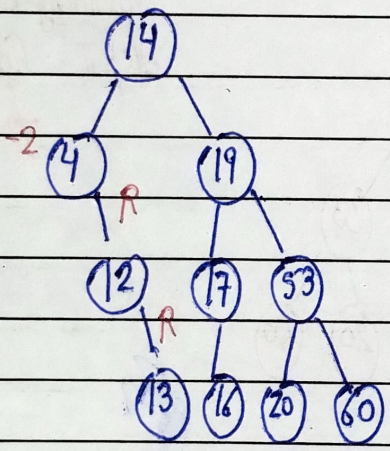


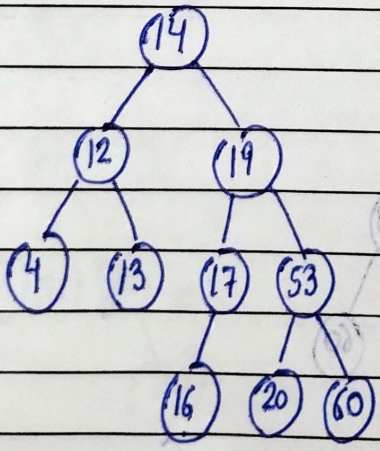
Date : / /
Page No.

Delete 7



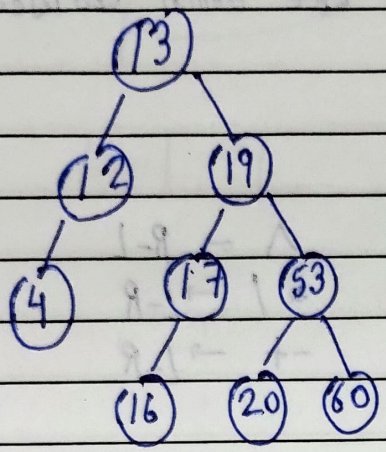
Delete 11



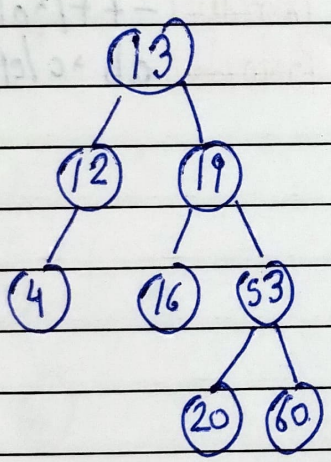


Date : / /
Page No.

Delete 14



Delete 17



Time Complexity & Space Complexity

	Best case	Average case	Worst case
Insert	O(logn)	O(logn)	O(logn)
Delete	O(logn)	O(logn)	O(logn)
Search	O(1)	O(logn)	O(logn)
Traversal	O(logn)	O(logn)	O(logn)
Space	O(n)	O(n)	O(n)